

Blocco 12 - Capstone architettura DBMS e dashboard

Dalla specifica al sistema containerizzato completo

Relational Databases & SQL

30 min teoria + 90 min pratica

1. Capstone

L'obiettivo finale non è una singola query, ma un sistema piccolo, spiegabile e replicabile.

Obiettivo del blocco

Problema

Integrare tutto: Docker Compose, collector simulato, PostgreSQL, schema relazionale, viste SQL, query Metabase, dashboard e controlli.

Idea guida

Una soluzione professionale descrive architettura, dati, query, verifiche e limiti.

Cosa deve essere consegnato

- ▶ una descrizione dell'architettura;
- ▶ il comando per avviare i container;
- ▶ lo schema dati e il ruolo delle tabelle;
- ▶ almeno cinque query per dashboard;
- ▶ una dashboard Metabase costruita con quelle query;
- ▶ controlli di qualità e performance;
- ▶ una breve spiegazione dei limiti.

Elemento	Descrizione
Collector	genera ticket simulati e li invia al DBMS
PostgreSQL	conserva raw, curated, eventi e viste dashboard
Metabase	interroga PostgreSQL e presenta card e dashboard
SQL	definisce schema, trasformazioni, metriche e controlli
Docker Compose	rende l'ambiente riproducibile

Da richiesta informale a specifica

Richiesta iniziale

“Voglio vedere come stanno andando i ticket e capire dove intervenire.”

Questa frase non è ancora una specifica: mancano popolazione, metriche, granularità, periodo, filtri, visualizzazioni e controlli.

- ▶ popolazione: ticket nello schema `ticketing`;
- ▶ metrica 1: ticket aperti, chiusi e violazioni SLA;
- ▶ metrica 2: trend giornaliero e backlog;
- ▶ metrica 3: ranking categorie e tempi medi;
- ▶ filtri: data apertura, priorità, regione, categoria;
- ▶ output: dashboard Metabase con query SQL versionate.

2. Avvio del sistema

La riproducibilità parte dai comandi, non dalle slide.

Comandi di avvio

```
# Dalla root della repository studenti
docker compose -f docker-compose.ticketing.yml up -d

# Controllo container
docker compose -f docker-compose.ticketing.yml ps

# Log del collector simulato
docker logs -f rdsql-ticket-collector
```

Servizio	Accesso
PostgreSQL Database	host localhost, porta 5433
Utente/password	training
Metabase	training/training
	http://localhost:3000

Connessione psql

```
docker exec -it rdsq1-ticket-postgres \  
    psql -U training -d training
```

```
-- dentro psql
```

```
SET search_path TO ticketing;
```

```
SELECT count(*) FROM support_tickets;
```

Connessione Metabase a PostgreSQL

- ▶ aprire `http://localhost:3000`;
- ▶ creare l'account iniziale;
- ▶ aggiungere database PostgreSQL;
- ▶ host: `postgres` se Metabase gira nel Compose;
- ▶ porta: `5432` dentro la rete Docker;
- ▶ database, utente e password: `training`.

3. Query della dashboard

Ogni card deve poter essere spiegata e verificata.

Set minimo di card

- ① KPI complessivi;
- ② serie giornaliera aperti/risolti/backlog;
- ③ distribuzione priorità-stato;
- ④ tempo medio di risoluzione per priorità e regione;
- ⑤ ranking categorie;
- ⑥ tabella ticket aperti per drill-down.

Card 1: KPI

```
SELECT count(*) AS total_tickets,  
       count(*) FILTER (WHERE closed_at IS NULL) AS open_tickets,  
       count(*) FILTER (WHERE closed_at IS NOT NULL) AS closed_tickets,  
       count(*) FILTER (WHERE sla_breached) AS sla_breached_tickets  
FROM ticketing.dashboard_ticket_base;
```

Card 2: trend giornaliero

```
SELECT day,  
       opened_tickets,  
       resolved_tickets,  
       backlog_delta  
FROM ticketing.dashboard_daily_flow  
ORDER BY day;
```


Card 3: ranking categorie

```
SELECT category,  
       count(*) AS tickets,  
       rank() OVER (ORDER BY count(*) DESC, category) AS category_rank  
FROM ticketing.dashboard_ticket_base  
GROUP BY category  
ORDER BY category_rank, category;
```

Card 4: ticket aperti

```
SELECT ticket_id, source_code, external_ticket_id,  
       opened_at, priority, category, region,  
       customer_segment, channel, subject,  
       sla_due_at, sla_breached  
FROM ticketing.dashboard_ticket_base  
WHERE closed_at IS NULL  
ORDER BY priority DESC, opened_at;
```

Costruzione dashboard in Metabase

- ▶ creare ogni query come “Native query”;
- ▶ salvare la query con nome descrittivo;
- ▶ scegliere visualizzazione coerente con la granularità;
- ▶ aggiungere le domande alla dashboard;
- ▶ configurare filtri comuni solo dove hanno lo stesso significato.

4. Verifica

Un sistema è credibile se può essere controllato.

- ▶ raw event e ticket curati sono allineati?
- ▶ i ticket aperti nella dashboard coincidono con la tabella base?
- ▶ i conteggi aggregati tornano al totale?
- ▶ le violazioni SLA sono coerenti con `sla_due_at`?
- ▶ il collector sta aggiungendo nuove righe?

Controllo raw-curated

```
SELECT (SELECT count(*) FROM ticketing.support_tickets_raw) AS raw_events,  
(SELECT count(*) FROM ticketing.support_tickets) AS curated_tickets,  
(SELECT count(*)  
  FROM ticketing.support_tickets_raw r  
  LEFT JOIN ticketing.support_tickets t  
    ON t.external_ticket_id = r.external_ticket_id  
 WHERE t.ticket_id IS NULL) AS raw_without_curated_ticket;
```

Controllo performance

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT ticket_id, opened_at, priority, region, status
FROM ticketing.support_tickets
WHERE status IN ('open', 'assigned', 'waiting_customer')
ORDER BY opened_at DESC, ticket_id DESC;
```

Limiti da dichiarare

- ▶ il collector è simulato, non legge una vera API;
- ▶ il dataset è piccolo, quindi i benefici degli indici possono non essere evidenti;
- ▶ Metabase usa una configurazione locale semplificata;
- ▶ non sono trattati autenticazione, backup e sicurezza di produzione;
- ▶ l'obiettivo è didattico: rendere replicabile il ragionamento architetturale.

5. Presentazione finale

Ogni gruppo deve difendere sia il modello sia le query.

Struttura della presentazione

- ➊ architettura e responsabilità dei container;
- ➋ modello dati e scelte raw/curated;
- ➌ viste SQL esposte alla dashboard;
- ➍ card Metabase e significato delle metriche;
- ➎ controlli dati e performance;
- ➏ limiti e possibili estensioni.

- ▶ chiarezza della specifica;
- ▶ coerenza tra architettura, schema e query;
- ▶ correttezza SQL;
- ▶ qualità delle spiegazioni;
- ▶ presenza di controlli;
- ▶ capacità di discutere trade-off.

Estensioni possibili

- ▶ aggiungere un secondo DB per separare operational e analytics;
- ▶ introdurre una tabella eventi più ricca;
- ▶ aggiungere refresh pianificato di viste materializzate;
- ▶ creare filtri Metabase per priorità, regione e periodo;
- ▶ simulare errori raw non caricabili nel curated.

Organizzazione dei 90 minuti

- ▶ 20 min avvio stack e controllo schema;
- ▶ 25 min query e card Metabase;
- ▶ 20 min dashboard e filtri;
- ▶ 15 min controlli;
- ▶ 10 min presentazione sintetica.

- ▶ il capstone collega SQL, DBMS, container e dashboard;
- ▶ una dashboard affidabile nasce da un modello dati affidabile;
- ▶ la consegna finale deve essere replicabile, spiegabile e verificabile.